Day01 项目导学与开发环境搭建

> 课程：交通时空大数据分析与可视化系统（Python 版） ｜ 第 1 天（8 课时）
> 主题：理解项目背景、掌握数据全貌、搭建开发环境、写出第一个可运行脚本

一、本日目标

1. 理解信令数据是什么、能做什么，建立项目整体认知；
2. 掌握 7 个数据文件的字段、编码、行数、关联关系；
3. 完成 Python 开发环境搭建，验证 numpy/pandas/matplotlib 可用；
4. 掌握 Jupyter Notebook 与 VS Code 两种开发环境的使用；
5. 复习 Python 核心语法（重点讲与 C/Java 的差异）；
6. 理解模块化与包管理机制；
7. 写出第一个"读 CSV → 统计 → 保存"的可运行脚本，建立工程习惯。

二、知识要点

- 信令数据 = 手机与基站通信时产生的位置记录，是"人移动轨迹"的代理数据；
- 本项目用 Python 三件套替代原 Hadoop/Spark/Kafka/MySQL/echarts 技术栈；
- 开发环境统一用 Python 3.10+，包管理用 pip；
- 工程习惯：数据/代码/结果分目录、函数封装、参数集中、逐行注释。

课时1：项目背景与业务目标

1.1 课时目标

理解"我们在做什么、为什么做、做成什么样"。

1.2 知识点深度讲解

1.2.1 什么是信令数据

手机在开机状态下，会持续与周围基站通信。每次通信（通话、短信、上网、位置更新、切换基站）都会在运营商侧留下一条记录，包含：
- **谁**：用户标识（imsi，国际移动用户识别码，类似手机身份证）；
- **何时**：通信发生的时间戳；
- **在哪**：连接的基站编号（lac_id + cell_id），基站有固定经纬度。

因此，信令数据本质上是**"人在什么时间出现在什么基站附近"的序列记录**。通过基站经纬度，就能把基站编号翻译成空间坐标，从而还原人的移动轨迹。

> 注意：信令数据不是 GPS 数据，精度取决于基站密度。城市密集区基站间距约 100-500 米，郊区可能达数公里。本项目数据来自沈阳，基站密度较高。

1.2.2 信令数据能做什么

基于人的移动轨迹，可以分析：
- **出行与驻留**：人什么时候在移动、什么时候在停留；
- **出行目的**：停留的地方是家、商场还是景点；
- **出行方式**：移动时坐地铁、公交、小汽车还是步行；
- **实时人流热力**：某个商圈/区域当前有多少人；
- **通勤规律**：早晚高峰、职住分离等城市交通特征。

这些分析结果可用于：城市规划、交通调度、商业选址、应急管理、疫情防控等。

1.2.3 本项目的业务目标

本项目要实现一个完整的"交通时空大数据分析系统"，包含 6 大模块：



1.2.4 技术栈选型说明

原项目使用 Hadoop + Hive + Spark + Kafka + MySQL + echarts，是典型的大数据技术栈。但对于教学场景，这套技术栈有几个问题：

1. **环境搭建复杂**：Hadoop/Spark/Kafka 需要分布式环境或伪分布式，配置繁琐；
2. **学习曲线陡峭**：学生需要同时掌握 Java/Scala、SQL、前端等多种技术；
3. **聚焦不在算法**：大量时间花在环境配置和框架 API 上，核心业务逻辑反而被稀释。

因此，本课程改用 **Python + NumPy + Pandas + Matplotlib**：

- 环境简单：pip install 即可；

- 聚焦算法：手写 k-means、Haversine 距离、滑窗统计，锻炼算法能力；
- 可视化强大：Matplotlib 能画出所有需要的图表，包括 3D 轨迹；
- 性能足够：53 万行数据在单机上秒级处理，教学场景完全够用。

> 注意：本项目不使用 sklearn（k-means 手写）、不使用 geopandas（经纬度直接用 Haversine 公式），目的是让学生理解算法本质，而不是调包。

1.3 项目实现步骤

本课时不写代码，重点是建立整体认知。后续 13 天会逐步实现每个模块。

1.4 当堂练习

1. 用自己的话解释"信令数据"和"GPS 数据"的区别；
2. 画出本项目 6 大模块的数据流图（不看上面的图，自己画）；
3. 思考：如果只有 1 万行数据，还需要用 Hadoop/Spark 吗？为什么？

1.5 课时小结

- 信令数据 = 谁 + 何时 + 在哪（基站），是人的移动轨迹代理数据；
- 本项目 6 大模块：清洗 → 出行驻留 → 目的/方式 → 实时热力 → 可视化；
- 技术栈用 Python 三件套，环境简单、聚焦算法、性能足够。

课时2：数据集全景解读

2.1 课时目标

掌握 7 个数据文件的字段、编码、行数、样例数据、关联关系。

2.2 知识点深度讲解

2.2.1 为什么要先读数据

动手写代码前，必须先搞清楚：
- 数据有多少行、多少列；
- 每列是什么含义、什么类型；
- 数据编码是什么（UTF-8 还是 GBK）；
- 数据之间怎么关联（哪个字段是关联键）；
- 数据有没有缺失、异常。

> 这是数据科学的"黄金法则"：Garbage In, Garbage Out。不了解数据就写代码，等于盲人摸象。

2.2.2 7 个数据文件详解

文件1：原始数据.csv（核心输入）

- 行数：约 53 万行（实际 532,828 行）
- 编码：UTF-8
- 列数：10 列，无表头
- 字段：

列序	字段名	类型	含义	样例
1	timestamp	int	毫秒时间戳（13位）	1538505600000
2	imsi	str	用户标识（国际移动用户识别码）	460000095007329090
3	lac_id	str	位置区编码	16789
4	cell_id	str	小区编号	67567924
5	phone	str	手机号（脱敏）	138****1234
6	timestamp1	int	备用时间戳	1538505600
7	tmp0	str	临时字段0	-
8	tmp1	str	临时字段1	-
9	nid	str	网络标识	-
10	npid	str	网络点标识	-

> 关键注意：`lac_id` 和 `cell_id` 必须按字符串读取！如果按数字读取，pandas 会推断为 float（如 `16789.0`），与基站表的 `laci`（形如 `16789-67567924`）拼接时对不上，导致关联后 0 行。这是本项目最容易踩的坑之一。
>
> `imsi` 也必须按字符串读取，否则大整数会丢失精度（Python int 虽然不溢出，但 pandas 推断为 int64 后，imsi 前导零会丢失，且与其他表关联时类型不匹配）。

文件2：基站经纬度数据.csv（空间补全用）

- 行数：49,954 行
- 编码：UTF-8

- 列数：3 列，无表头
- 字段：

列序	字段名	类型	含义	样例
1	longitude	float	经度	123.4119873
2	latitude	float	纬度	41.8078804
3	laci	str	基站唯一标识（lac_id-cell_id）	16789-67567924

> `laci` 是关联键，由 `lac_id` + `-` + `cell_id` 拼接而成。原始数据没有经纬度，需要通过这个键关联基站表补全。

文件3：商业区数据.csv（实时热力用）
- 行数：104 行
- 编码：UTF-8
- 列数：3 列，无表头
- 字段：经度、纬度、商圈名
- 样例商圈：太原街商圈、中街商圈、沈阳北站、棋盘山风景区、沈阳世博园等

文件4：出行目的静态数据.csv（目的识别用）
- 行数：1,273 行
- 编码：**GBK**（重要！不是 UTF-8）
- 列数：4 列，无表头
- 字段：经度、纬度、目的类别、目的名
- 样例：`123.457628,41.722539,居住,万科·明天广场`

> GBK 编码的文件如果用 UTF-8 读取，中文会乱码。必须用 `encoding='gbk'`。

文件5：出行方式静态数据.csv（方式识别用）
- 行数：968 行
- 编码：**GBK**
- 列数：5 列，无表头
- 字段：方式类型、经度、纬度、...（具体列在 Day11 详解）

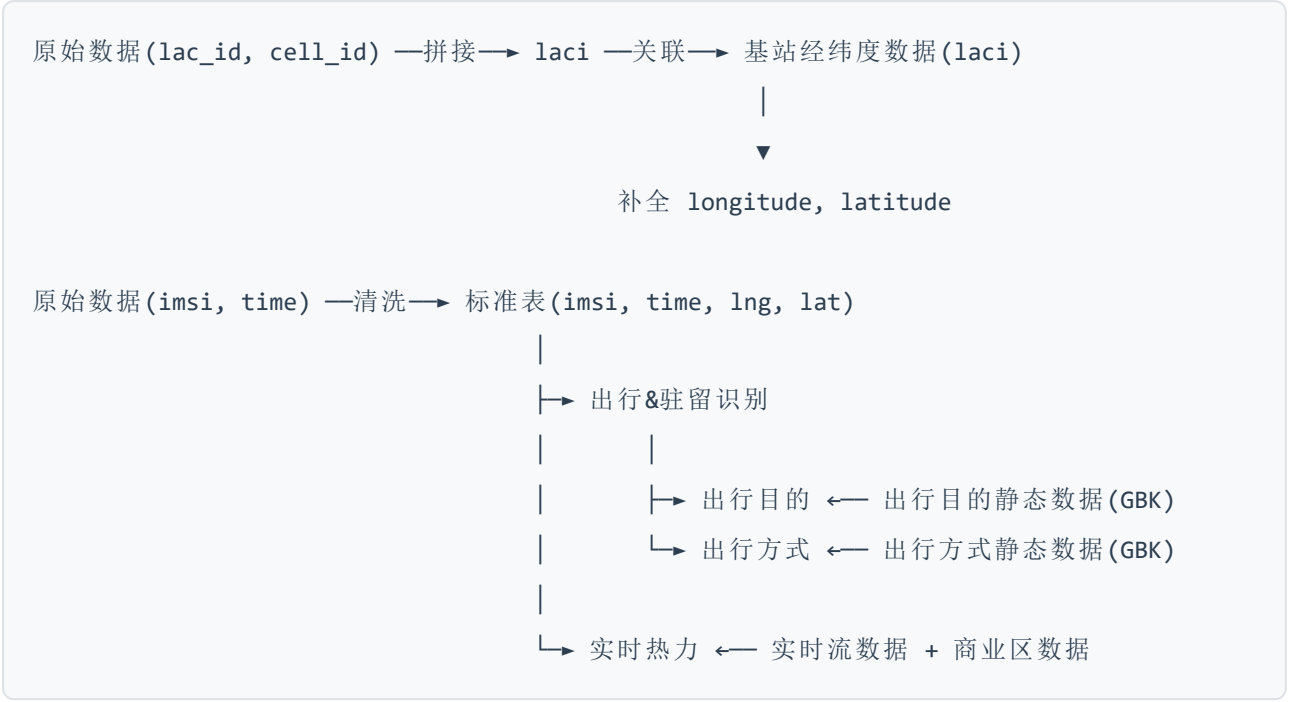
文件6：实时流数据.csv（实时热力用）
- 行数：256,212 行
- 编码：UTF-8
- 列数：3 列，无表头

- 字段：时间、imsi、经度、纬度
- 样例：`20181003000003,460000095090934465,123.4119873,41.8078804`

> 实时流数据已经有经纬度（不需要关联基站表），时间格式是 `YYYYMMDDHHMMSS`（14位字符串）。

文件7：hive模块静态数据.csv（参考用）
- 行数：225 行
- 编码：UTF-8
- 列数：2 列，无表头
- 字段：区域名、laci
- 用途：原项目 Hive 模块用，Python 版可参考但不必须使用。

2.2.3 数据间关联关系



2.2.4 目标日期与城市

- 目标日期：2018-10-03（`20181003`）
- 城市：沈阳（经度范围约 122°~123.9°，纬度范围约 41.1°~41.9°）

2.3 项目实现步骤

本课时重点是"读数据、看数据"，不做清洗。用 Python 读取每个文件，打印行数、列名、前几行。

2.4 完整代码（逐行注释）

```
# ============================================================
# 课时2：数据集全景解读
# 功能：读取7个数据文件，打印基本信息，建立数据认知
# ============================================================

import pandas as pd   # 导入pandas，用于读取CSV和数据探查

# 数据根目录（注意路径中有括号和反斜杠，用r前缀表示原始字符串）
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统(\09.交通时空大数据分析系统\B-项目数据"

# ------------------------------------------------------------
# 1. 读取原始数据
# ------------------------------------------------------------
# header=None：CSV没有表头，pandas不要把第一行当列名
# names=...：手动指定10列的列名
# dtype=...：指定关键列按字符串读取，避免pandas推断为数字导致精度丢失
# nrows=5：只读前5行，快速查看样例（不读全部，节省时间）
raw_cols = ["timestamp", "imsi", "lac_id", "cell_id", "phone",
            "timestamp1", "tmp0", "tmp1", "nid", "npid"]
raw = pd.read_csv(DATA_DIR + r"\原始数据.csv",
                  header=None, names=raw_cols,
                  dtype={"imsi": str, "lac_id": str, "cell_id": str,
                         "phone": str, "nid": str},
                  nrows=5, encoding="utf-8")
print("=== 原始数据（前5行）===")
print(raw)
print("列名:", raw.columns.tolist())
print()

# ------------------------------------------------------------
# 2. 读取基站经纬度数据
# ------------------------------------------------------------
bs = pd.read_csv(DATA_DIR + r"\基站经纬度数据.csv",
                 header=None, names=["longitude", "latitude", "laci"],
                 encoding="utf-8")
```

```
print("=== 基站经纬度数据 ===")
print("行数:", len(bs), " 列数:", bs.shape[1])
print(bs.head(3))
print("laci样例:", bs["laci"].iloc[0])
print()

# ------------------------------------------------------------
# 3. 读取商业区数据
# ------------------------------------------------------------
area = pd.read_csv(DATA_DIR + r"\商业区数据.csv",
                   header=None, names=["longitude", "latitude", "name"],
                   encoding="utf-8")
print("=== 商业区数据 ===")
print("行数:", len(area))
print("商圈清单:", area["name"].tolist()[:10])
print()

# ------------------------------------------------------------
# 4. 读取出行目的静态数据（GBK编码！）
# ------------------------------------------------------------
purpose_poi = pd.read_csv(DATA_DIR + r"\出行目的静态数据.csv",
                          header=None, encoding="gbk")   # 注意：gbk不是utf-8
print("=== 出行目的静态数据 ===")
print("行数:", len(purpose_poi), " 列数:", purpose_poi.shape[1])
print("第一行样例:", purpose_poi.iloc[0].tolist())
print()

# ------------------------------------------------------------
# 5. 读取出行方式静态数据（GBK编码！）
# ------------------------------------------------------------
mode_poi = pd.read_csv(DATA_DIR + r"\出行方式静态数据.csv",
                       header=None, encoding="gbk")
print("=== 出行方式静态数据 ===")
print("行数:", len(mode_poi), " 列数:", mode_poi.shape[1])
print("第一行样例:", mode_poi.iloc[0].tolist())
print()

# ------------------------------------------------------------
```

```
# 6. 读取实时流数据
# ------------------------------------------------------------
flow = pd.read_csv(DATA_DIR + r"\实时流数据.csv",
                   header=None, names=["time", "imsi", "longitude", "latitude"],
                   dtype={"imsi": str, "time": str},
                   encoding="utf-8")
print("=== 实时流数据 ===")
print("行数:", len(flow))
print(flow.head(3))
print("时间样例:", flow["time"].iloc[0], "（格式：YYYYMMDDHHMMSS）")
print()

# ------------------------------------------------------------
# 7. 读取hive模块静态数据
# ------------------------------------------------------------
hive = pd.read_csv(DATA_DIR + r"\hive模块静态数据.csv",
                   header=None, names=["area_name", "laci"],
                   encoding="utf-8")
print("=== hive模块静态数据 ===")
print("行数:", len(hive))
print(hive.head(3))
```

2.5 运行结果与解读

运行上述代码，应看到：
- 原始数据前5行，确认10列字段；
- 基站数据 49,954 行，laci 形如 `16789-67567924`；
- 商业区 104 个，含太原街、中街等；
- 目的静态数据 1,273 行，GBK 编码中文正常显示；
- 方式静态数据 968 行；
- 实时流 256,212 行，时间格式 14 位字符串；
- hive 数据 225 行。

> 如果目的/方式数据中文乱码，说明编码不对，检查是否用了 `encoding='gbk'`。

2.6 当堂练习

1. 修改代码，统计原始数据的总行数（去掉 `nrows=5`），并统计不同 imsi 的数量；

2. 查看基站经纬度数据的经纬度范围（min/max），确认是否在沈阳范围内；

3. 思考：为什么原始数据没有经纬度，而实时流数据有经纬度？

2.7 课时小结

- 7 个数据文件各有用途，核心是原始数据 + 基站表；
- 两个关键坑：`lac_id/cell_id/imsi` 必须按字符串读取；目的/方式数据是 GBK 编码；
- 数据关联键：原始数据的 `lac_id-cell_id` 拼接成 `laci`，关联基站表补经纬度。

课时3：Python环境搭建与验证

3.1 课时目标

完成 Python 开发环境搭建，验证 numpy/pandas/matplotlib 可用。

3.2 知识点深度讲解

3.2.1 Python版本选择

本课程使用 **Python 3.10+**。为什么不选 Python 2?
- Python 2 已于 2020 年停止维护，不再有安全更新；
- Python 3 有更好的 Unicode 支持（中文处理不踩坑）；
- 新版 numpy/pandas/matplotlib 已不再支持 Python 2。

> 注意：Python 3.10+ 有一些新语法（如 `match-case`、联合类型 `X | Y`），但本课程主要用基础语法，3.8+ 也能运行。建议统一用 3.10 或 3.11。

3.2.2 两种安装方式对比

方式1：Anaconda（推荐新手）
- 优点：自带 numpy/pandas/matplotlib 等 200+ 科学计算包，不用逐个装；自带 conda 包管理器和环境管理；Jupyter Notebook 开箱即用。
- 缺点：安装包大（约 500MB），占用空间多；包版本可能不是最新。
- 下载：https://www.anaconda.com/download

方式2：原生 Python + pip（推荐有基础的学生）
- 优点：轻量（安装包约 30MB），只装需要的包；pip 是 Python 官方包管理器，包最新。
- 缺点：需要手动装 numpy/pandas/matplotlib；环境管理需要自己处理。
- 下载：https://www.python.org/downloads/

> 本课程两种方式都支持。如果学生电脑空间紧张，用原生 Python + pip；如果想省事，用 Anaconda。

3.2.3 pip 换源（国内必做）

pip 默认从 PyPI（pypi.org）下载包，国内访问慢。换国内镜像源：

```
# 临时换源（单次安装）
pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple

# 永久换源（推荐）
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

常用国内镜像源：
- 清华大学：https://pypi.tuna.tsinghua.edu.cn/simple
- 阿里云：https://mirrors.aliyun.com/pypi/simple
- 中科大：https://pypi.mirrors.ustc.edu.cn/simple

3.2.4 虚拟环境（可选但推荐）

为什么需要虚拟环境？
- 不同项目可能需要不同版本的包（如项目A用 pandas 1.5，项目B用 pandas 2.0）；
- 虚拟环境把每个项目的依赖隔离开，互不影响；
- 避免"在我电脑上能跑，在你电脑上跑不了"的问题。

```
# 创建虚拟环境
python -m venv venv

# 激活虚拟环境（Windows）
venv\Scripts\activate

# 激活后，命令行前面会出现 (venv) 标识
# 此时 pip install 装的包只在这个虚拟环境里

# 退出虚拟环境
deactivate
```

> 本课程为了简化，不强制使用虚拟环境。但建议学生了解这个概念，工作后会用到。

3.3 项目实现步骤

1. 安装 Python（Anaconda 或原生）；
2. 换 pip 国内源；
3. 安装 numpy/pandas/matplotlib；
4. 验证安装成功。

3.4 完整代码（验证脚本）

```
# ============================================================
# 课时3：Python环境验证脚本
# 功能：验证 Python 版本、numpy/pandas/matplotlib 是否可用
# 运行方式：python check_env.py
# ============================================================

import sys   # sys模块提供Python解释器的相关信息

# ------------------------------------------------------------
# 1. 验证Python版本
# ------------------------------------------------------------
print("=" * 50)
print("Python环境验证")
print("=" * 50)
print(f"Python版本：{sys.version}")   # sys.version包含完整版本信息
print(f"Python可执行文件路径：{sys.executable}")   # 当前用的是哪个python

# 检查版本是否 >= 3.10
# sys.version_info是一个元组，如 (3, 11, 4, 'final', 0)
major, minor = sys.version_info[0], sys.version_info[1]
if major >= 3 and minor >= 10:
    print(f"版本检查通过：Python {major}.{minor} >= 3.10")
else:
    print(f"警告：Python {major}.{minor} < 3.10，建议升级")
print()

# ------------------------------------------------------------
# 2. 验证numpy
# ------------------------------------------------------------
try:
    import numpy as np   # 导入numpy，惯例别名np
    print(f"numpy版本：{np.__version__}")
    # 简单测试：创建一个数组并计算
    arr = np.array([1, 2, 3, 4, 5])
    print(f"numpy测试：array={arr.tolist()}, mean={np.mean(arr):.2f}")
    print("numpy 安装成功 ✓")
```

```
except ImportError as e:
    print(f"numpy 安装失败 ✗: {e}")
    print("请运行: pip install numpy")
print()

# ------------------------------------------------------------
# 3. 验证pandas
# ------------------------------------------------------------
try:
    import pandas as pd   # 导入pandas，惯例别名pd
    print(f"pandas版本: {pd.__version__}")
    # 简单测试：创建一个DataFrame
    df = pd.DataFrame({"name": ["Alice", "Bob"], "age": [25, 30]})
    print(f"pandas测试: DataFrame形状={df.shape}")
    print("pandas 安装成功 ✓")
except ImportError as e:
    print(f"pandas 安装失败 ✗: {e}")
    print("请运行: pip install pandas")
print()

# ------------------------------------------------------------
# 4. 验证matplotlib
# ------------------------------------------------------------
try:
    import matplotlib   # 先导入matplotlib主模块
    print(f"matplotlib版本: {matplotlib.__version__}")
    # 设置后端为Agg（非交互式后端，不需要图形界面也能保存图片）
    # 这在服务器/无窗口环境下必须设置
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt   # 导入pyplot子模块，惯例别名plt
    # 简单测试：画一条线并保存
    fig, ax = plt.subplots(figsize=(4, 3))
    ax.plot([1, 2, 3], [1, 4, 9])
    ax.set_title("test")
    fig.savefig("test_plot.png")   # 保存图片到当前目录
    plt.close(fig)   # 关闭图，释放内存
    print("matplotlib 安装成功 ✓（测试图已保存为 test_plot.png）")
except ImportError as e:
```

```
    print(f"matplotlib 安装失败 ✗: {e}")
    print("请运行: pip install matplotlib")
print()

# ------------------------------------------------------------
# 5. 总结
# ------------------------------------------------------------
print("=" * 50)
print("环境验证完成。如果全部显示 ✓，可以开始上课。")
print("=" * 50)
```

3.5 运行结果与解读

运行 `python check_env.py`，应看到：
- Python 版本（3.10+）；
- numpy 版本（如 1.26.x），测试通过；
- pandas 版本（如 2.1.x），测试通过；
- matplotlib 版本（如 3.8.x），测试通过，生成 test_plot.png。

> 如果某个包导入失败，按照提示运行 `pip install 包名` 安装。
>
> **重要提示**：本课程环境中，默认 `python` 命令可能指向沙箱 runtime（无 numpy）。装好三件套的解释器路径可能是 `D:\soft\python\python.exe`。如果 `python check_env.py` 报 numpy 找不到，检查 `sys.executable` 输出的路径，用正确的解释器运行。

3.6 当堂练习

1. 在自己电脑上安装 Python（Anaconda 或原生），运行验证脚本，截图结果；
2. 用 `pip list` 查看已安装的包，确认 numpy/pandas/matplotlib 的版本；
3. 尝试创建一个虚拟环境，在虚拟环境里安装 pandas，然后退出虚拟环境，观察系统 Python 里有没有 pandas。

3.7 课时小结

- Python 3.10+，用 pip 安装 numpy/pandas/matplotlib；
- 国内必做 pip 换源；
- 用验证脚本确认环境可用，注意 `sys.executable` 确认用的是哪个解释器。

课时4：Jupyter Notebook 与 VS Code

4.1 课时目标

掌握两种开发环境的使用，选择适合自己的工具。

4.2 知识点深度讲解

4.2.1 Jupyter Notebook（适合探索性分析）

Jupyter Notebook 是一个交互式笔记本，特点：
- **单元格（Cell）**：代码和文档混排，每个单元格独立运行；
- **即时反馈**：运行一个单元格立刻看到结果，不用写完整脚本；
- **Markdown 支持**：可以在代码之间插入文字、公式、图片，形成可复现的分析报告；
- **适合场景**：数据探查、算法调试、教学演示、分析报告。

常用快捷键（命令模式下，按 Esc 进入命令模式）：
- `A`：在上方插入单元格
- `B`：在下方插入单元格
- `DD`：删除当前单元格
- `M`：单元格转为 Markdown
- `Y`：单元格转为代码
- `Shift+Enter`：运行当前单元格并跳到下一个
- `Ctrl+Enter`：运行当前单元格（不跳转）

启动方式：

```
# 启动Jupyter Notebook（会在浏览器打开）
jupyter notebook


# 或者启动JupyterLab（更现代的界面，推荐）
jupyter lab
```

4.2.2 VS Code（适合工程开发）

Visual Studio Code 是一个轻量级代码编辑器，特点：
- **强大的代码编辑**：语法高亮、智能补全、代码跳转、重构；
- **集成终端**：编辑器里直接运行命令，不用切换窗口；
- **调试功能**：断点、单步执行、变量查看，调试复杂代码必备；
- **Git 集成**：版本控制可视化；
- **适合场景**：模块化开发、多文件项目、调试、工程化。

必备插件:
- Python（微软官方）：Python 语言支持、调试、Linting；
- Pylance：快速、功能强大的 Python 语言服务器；
- Jupyter：在 VS Code 里直接运行 .ipynb 文件；
- Chinese (Simplified)：中文界面（可选）。

常用快捷键:
- `Ctrl+``：打开/关闭集成终端` - Ctrl+Shift+P`：打开命令面板` - F5`：开始调试` - F9`：切换断点` - Ctrl+/`：注释/取消注释` - Alt+↑/↓：上下移动当前行

4.2.3 两种工具怎么选

维度	Jupyter Notebook	VS Code
学习曲线	低，打开即用	中，需要熟悉快捷键和插件
适合阶段	数据探查、算法调试	模块化开发、多文件项目
代码组织	单元格，顺序执行	文件+函数，结构化
调试能力	弱（print 调试为主）	强（断点+单步+变量查看）
教学演示	好（即时反馈+文档混排）	一般（需要切换终端看输出）

> 建议：本课程前半段（Day01-Day06）用 Jupyter Notebook 做探索和调试，后半段（Day07-Day14）用 VS Code 做模块化开发。两者结合，各取所长。

4.2.4 项目目录结构规范

一个规范的项目目录结构：

```
交通时空大数据分析系统/
├── data/                    # 数据目录（不入库，太大）
│   ├── raw/                 # 原始数据
│   └── processed/           # 清洗后的数据
├── project_code/            # 代码目录
│   ├── config.py            # 配置文件（路径、阈值、参数）
│   ├── utils.py             # 工具函数（haversine、setup_zh等）
│   ├── data_clean.py        # 数据清洗模块
│   ├── tr_stage1.py         # 出行驻留-阶段1
│   ├── tr_stage2.py         # 出行驻留-阶段2
│   ├── tr_stage3.py         # 出行驻留-阶段3
│   ├── purpose.py           # 出行目的识别
│   ├── mode.py              # 出行方式识别
│   ├── heatmap.py           # 实时热力
│   ├── visualize.py         # 可视化
│   └── main.py              # 主入口（一键跑通全流程）
├── result/                  # 结果目录
│   ├── cleaned_data.csv     # 清洗结果
│   ├── stage3_label.csv     # 标签结果
│   ├── purpose_result.csv   # 目的结果
│   ├── mode_result.csv      # 方式结果
│   ├── heatmap_result.csv   # 热力结果
│   └── figures/             # 图片目录
│       ├── fig_stage3_pie.png
│       ├── fig_purpose_bar.png
│       └── ...
├── notebooks/               # Jupyter笔记本（探索用）
│   ├── 01_data_explore.ipynb
│   └── ...
├── requirements.txt         # 依赖清单
└── README.md                # 项目说明
```

> 为什么要分目录？数据/代码/结果分离，便于管理和版本控制。数据目录太大不入库，代码和结果可以入库。

4.3 项目实现步骤

1. 创建项目目录结构；
2. 在 Jupyter Notebook 里做数据探查；
3. 在 VS Code 里写模块化代码。

4.4 完整代码（创建目录结构）

```
# ============================================================
# 课时4：创建项目目录结构
# 功能：自动创建规范的项目目录
# ============================================================

import os   # os模块提供文件系统操作功能

# 项目根目录
PROJECT_ROOT = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版"

# 要创建的目录列表
dirs = [
    "project_code",       # 代码目录
    "result",             # 结果目录
    "result/figures",     # 图片目录
    "notebooks",          # Jupyter笔记本目录
]

# 逐个创建目录
for d in dirs:
    full_path = os.path.join(PROJECT_ROOT, d)   # 拼接完整路径
    # exist_ok=True：如果目录已存在，不报错
    os.makedirs(full_path, exist_ok=True)
    print(f"创建目录：{full_path}")

print("\n目录结构创建完成！")
print("请将数据文件放入 data/ 目录，代码放入 project_code/ 目录。")
```

4.5 运行结果与解读

运行后，项目根目录下会出现 `project_code/`、`result/`、`result/figures/`、`notebooks/` 四个目录。

4.6 当堂练习

1. 在自己电脑上用 Jupyter Notebook 打开，创建一个笔记本，运行 `print("Hello, 交通时空大数据!")`；
2. 在 VS Code 里打开项目目录，安装 Python 插件，创建一个 `test.py` 文件并运行；
3. 思考：为什么 Jupyter Notebook 里变量在单元格之间可以共享，而 .py 脚本里不行？

4.7 课时小结

- Jupyter Notebook 适合探索和调试，VS Code 适合工程开发；
- 前半段用 Notebook，后半段用 VS Code，两者结合；
- 项目目录规范：数据/代码/结果分离，配置集中，函数封装。

课时5：Python基础速览（上）

5.1 课时目标

复习 Python 核心语法，重点讲与 C/Java 的差异，为后续代码打基础。

5.2 知识点深度讲解

5.2.1 变量与类型

Python 是**动态类型**语言，变量不需要声明类型，赋值即创建：

```
x = 10          # int
y = 3.14        # float
name = "Alice"  # str
flag = True     # bool
```

与 C/Java 的区别：
- C/Java：`int x = 10;` 必须声明类型；
- Python：`x = 10` 自动推断类型；
- Python 变量可以随时改变类型：`x = 10; x = "hello"`（合法，但不推荐）。

常用类型：

- `int`：整数（无大小限制，不会溢出）；
- `float`：浮点数（双精度）；
- `str`：字符串（Unicode，中文直接写）；
- `bool`：布尔值（`True` / `False`，首字母大写）；
- `None`：空值（类似 C 的 NULL、Java 的 null）。

类型转换：

```
int("123")      # 字符串转整数
float("3.14")   # 字符串转浮点数
str(123)        # 数字转字符串
bool(0)         # False，bool(非0)=True
```

5.2.2 运算符

- 算术：`+ - * / // % **`
- `/`：真除法（结果总是 float，如 `7/2=3.5`）；
- `//`：整除（向下取整，如 `7//2=3`，`-7//2=-4`）；
- `**`：幂运算（`2**3=8`）。
- 比较：`== != < > <= >=`（返回 bool）；
- 逻辑：`and or not`（不是 `&& || !`）；
- 成员：`in`（`3 in [1,2,3]` 返回 True）；
- 身份：`is`（比较对象身份，不是值，慎用）。

> 注意：Python 没有 `++` 和 `--` 运算符，用 `x += 1`。

5.2.3 条件语句

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 60:
    grade = "C"
else:
    grade = "D"
```

与 C/Java 的区别：
- 用 `elif` 不是 `else if`；
- 用缩进（4个空格）表示代码块，不是 `{}`；
- 条件后必须加冒号 `:`。

5.2.4 循环语句

for 循环（遍历可迭代对象）：

```
# 遍历列表
for i in [1, 2, 3]:
    print(i)

# 遍历range（range(5) = 0,1,2,3,4）
for i in range(5):
    print(i)

# 带索引遍历
for idx, val in enumerate(["a", "b", "c"]):
    print(idx, val)
```

while 循环：

```
count = 0
while count < 5:
    print(count)
    count += 1
```

> Python 的 for 循环是"for-each"风格，不是 C 的 `for(int i=0; i<n; i++)`。需要索引时用 `enumerate` 或 `range`。

break 和 continue：

- `break`：跳出循环；
- `continue`：跳过本次循环，继续下一次。

5.2.5 函数定义

```
def add(a, b=0):
    """计算两个数的和。

    Args:
        a: 第一个数
        b: 第二个数，默认0

    Returns:
        两数之和
    """
    return a + b

result = add(3, 5)      # 8
result = add(3)         # 3（用默认值）
result = add(b=5, a=3)  # 8（关键字参数，顺序无关）
```

要点：

- 用 `def` 定义，不是 `function` 或返回类型；
- 参数可以有默认值；
- 可以用关键字参数调用；
- 文档字符串（docstring）用三引号，是 Python 的规范写法；
- 函数可以返回多个值（实际是返回元组）：

`python def min_max(nums): return min(nums), max(nums) lo, hi = min_max([3, 1, 4, 1, 5]) # 解包`

5.2.6 异常处理

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"除零错误: {e}")
except (TypeError, ValueError) as e:
    print(f"类型或值错误: {e}")
else:
    print("没有异常时执行")
finally:
    print("无论是否异常都执行")
```

> 与 Java 的区别：Python 用 `except` 不是 `catch`，可以捕获多个异常类型，`else` 块在无异常时执行。

5.3 项目实现步骤

本课时复习语法，不直接操作项目数据。通过小练习巩固。

5.4 完整代码（语法练习）

```
# ============================================================
# 课时5：Python基础语法练习
# ============================================================

# 1. 变量与类型
x = 10
y = 3.14
name = "交通时空大数据"
print(f"x={x} (类型:{type(x).__name__}), y={y} (类型:{type(y).__name__})")
print(f"name={name} (类型:{type(name).__name__})")

# 2. 运算符
print(f"7/2={7/2}, 7//2={7//2}, 7%2={7%2}, 2**3={2**3}")
print(f"3 in [1,2,3] = {3 in [1,2,3]}")

# 3. 条件语句
score = 85
if score >= 90:
    print("优秀")
elif score >= 80:
    print("良好")
else:
    print("继续努力")

# 4. 循环
print("range(5):", end=" ")
for i in range(5):
    print(i, end=" ")
print()

# 带索引遍历
for idx, val in enumerate(["清洗", "聚类", "可视化"]):
    print(f"  步骤{idx+1}: {val}")

# 5. 函数
def haversine_approx(lng1, lat1, lng2, lat2):
```

```
    """简化版球面距离（用平面近似，仅用于语法练习）。"""
    import math
    dx = (lng2 - lng1) * 111  # 经度1度约111km
    dy = (lat2 - lat1) * 111  # 纬度1度约111km
    return math.sqrt(dx**2 + dy**2)

dist = haversine_approx(123.41, 41.80, 123.43, 41.80)
print(f"近似距离：{dist:.2f} km")

# 6. 异常处理
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"捕获异常：{e}")
finally:
    print("异常处理完成")
```

5.5 运行结果与解读

运行后应看到变量类型、运算结果、条件判断、循环输出、函数计算、异常捕获的完整演示。

5.6 当堂练习

1. 写一个函数，输入一个列表，返回列表中的最大值和最小值（用 return 返回两个值）；
2. 用 for 循环计算 1 到 100 的和（不能用 sum 函数）；
3. 写一个函数，输入年份，判断是否为闰年（能被4整除但不能被100整除，或能被400整除）。

5.7 课时小结

- Python 动态类型，赋值即创建，缩进表示代码块；
- for 循环是 for-each 风格，用 enumerate 获取索引；
- 函数用 def，支持默认参数、关键字参数、多返回值；
- 异常用 try/except/finally。

课时6：Python基础速览（下）

6.1 课时目标

掌握 Python 数据结构（列表/字典/集合/元组）、推导式、文件读写。

6.2 知识点深度讲解

6.2.1 列表（list）

列表是有序、可变的序列，类似 Java 的 ArrayList：

```
nums = [1, 2, 3, 4, 5]          # 创建
nums.append(6)                  # 末尾添加
nums.insert(0, 0)               # 指定位置插入
nums.remove(3)                  # 删除第一个值为3的元素
nums.pop()                      # 删除末尾元素并返回
nums[0]                         # 索引访问
nums[1:3]                       # 切片（含左不含右，返回[2,3]）
len(nums)                       # 长度
nums.sort()                     # 排序（原地修改）
sorted(nums)                    # 返回排序后的新列表
```

列表推导式（Python 特色，简洁高效）：

```
# 普通写法
squares = []
for x in range(10):
    squares.append(x ** 2)

# 推导式写法（等价，但更简洁）
squares = [x ** 2 for x in range(10)]

# 带条件的推导式
evens = [x for x in range(10) if x % 2 == 0]
```

> 推导式比 for 循环 + append 快，因为底层用 C 实现。本项目中会大量使用。

6.2.2 字典（dict）

字典是键值对集合，类似 Java 的 HashMap：

```
user = {"name": "Alice", "age": 25, "city": "沈阳"}  # 创建
user["name"]                                        # 访问
user["phone"] = "138****1234"                        # 添加/修改
user.get("email", "未知")                            # 安全访问（key不存在返回默认值）
user.keys()                                         # 所有键
user.values()                                       # 所有值
user.items()                                        # 所有键值对
"name" in user                                      # 键是否存在
```

字典推导式：

```
# 把列表变成字典（索引为键）
items = ["清洗", "聚类", "可视化"]
item_dict = {i: v for i, v in enumerate(items)}
# {0: "清洗", 1: "聚类", 2: "可视化"}
```

6.2.3 集合（set）

集合是无序、不重复的元素集合，类似 Java 的 HashSet：

```
s = {1, 2, 3, 3, 2}    # {1, 2, 3}（自动去重）
s.add(4)               # 添加
s.remove(1)            # 删除
1 in s                 # 成员判断（O(1)，比列表快）
```

集合运算：

```
a = {1, 2, 3}
b = {2, 3, 4}
a & b    # 交集 {2, 3}
a | b    # 并集 {1, 2, 3, 4}
a - b    # 差集 {1}
```

> 本项目中，集合常用于"去重"和"快速判断是否存在"，如判断某条记录是否在出行点集合中。

6.2.4 元组（tuple）

元组是有序、不可变的序列：

```
point = (123.41, 41.80)   # 创建（经纬度对）
point[0]                  # 访问（不能修改）
lng, lat = point          # 解包
```

> 元组不可变，适合表示"不应该被修改的数据"，如坐标对、函数多返回值。

6.2.5 文件读写

读取文本文件：

```
# 方式1：open + close（不推荐，容易忘记close）
f = open("data.csv", "r", encoding="utf-8")
content = f.read()
f.close()

# 方式2：with语句（推荐，自动关闭文件）
with open("data.csv", "r", encoding="utf-8") as f:
    content = f.read()          # 读取全部
    # 或逐行读取
    for line in f:
        print(line.strip())
```

写入文件：

```
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("第一行\n")
    f.write("第二行\n")
```

> `with` 语句是 Python 的上下文管理器，退出时自动调用 `f.close()`，即使发生异常也会关闭。这是 Python 文件操作的标准写法。

6.2.6 字符串常用操作

```
s = "  Hello, 交通时空大数据！  "
s.strip()                    # 去除首尾空白
s.lower()                    # 转小写
s.upper()                    # 转大写
s.split(",")                 # 按逗号分割成列表
s.replace("Hello", "Hi")     # 替换
s.startswith("  Hello")      # 是否以...开头
s.endswith("！  ")           # 是否以...结尾
f"结果是 {x}"                # f-string格式化（Python 3.6+）
```

> f-string 是 Python 3.6+ 的格式化方式，比 `%` 和 `.format()` 更直观，本项目统一用 f-string。

6.3 项目实现步骤

本课时复习数据结构和文件操作，为后续读取 CSV、处理数据做准备。

6.4 完整代码（数据结构练习）

```
# ============================================================
# 课时6：Python数据结构与文件操作练习
# ============================================================

# 1. 列表操作
records = ["imsi001", "imsi002", "imsi003"]
records.append("imsi004")
print(f"列表：{records}，长度：{len(records)}")
print(f"切片[1:3]：{records[1:3]}")

# 列表推导式：生成10个用户的imsi
imsis = [f"46000009500{i:04d}" for i in range(10)]
print(f"生成的imsi：{imsis[:3]}...")

# 2. 字典操作
user_info = {
    "imsi": "460000095007329090",
    "records": 156,
    "city": "沈阳"
}
print(f"用户imsi：{user_info['imsi']}，记录数：{user_info.get('records', 0)}")
user_info["purpose"] = "居住"   # 添加新键
print(f"用户信息键：{list(user_info.keys())}")

# 3. 集合去重
raw_imsis = ["imsi001", "imsi002", "imsi001", "imsi003", "imsi002"]
unique_imsis = set(raw_imsis)
print(f"原始：{len(raw_imsis)}个，去重后：{len(unique_imsis)}个")

# 4. 元组解包
point = (123.4119873, 41.8078804)
lng, lat = point
print(f"经度：{lng}，纬度：{lat}")

# 5. 文件读写（演示）
with open("demo_output.txt", "w", encoding="utf-8") as f:
```

```
    f.write("交通时空大数据分析系统\n")
    f.write("Python基础练习完成\n")

with open("demo_output.txt", "r", encoding="utf-8") as f:
    content = f.read()
print(f"文件内容:\n{content}")

# 6. 字符串操作
time_str = "20181003093000"
date = time_str[:8]       # 前8位是日期
hour = time_str[8:10]     # 9-10位是小时
minute = time_str[10:12]  # 11-12位是分钟
print(f"时间解析: 日期={date}, 小时={hour}, 分钟={minute}")
```

6.5 运行结果与解读

运行后演示列表/字典/集合/元组的基本操作、文件读写、字符串切片。特别注意时间字符串的切片，这是本项目后续处理时间字段的基础。

6.6 当堂练习

1. 用列表推导式生成 1 到 20 的平方数列表；
2. 用字典统计字符串 "abracadabra" 中每个字符出现的次数；
3. 写一个函数，读取一个文本文件，统计文件的行数和字符数。

6.7 课时小结

- 列表有序可变，字典键值对，集合无序去重，元组有序不可变；
- 推导式简洁高效，是 Python 特色；
- 文件用 with 语句，自动关闭；
- 字符串切片是时间字段处理的基础。

课时7：模块化与包管理

7.1 课时目标

理解 Python 的 import 机制，掌握模块化开发和 requirements.txt。

7.2 知识点深度讲解

7.2.1 为什么要模块化

当代码超过几百行，全部写在一个文件里会：
- 难以维护（找一个函数要翻半天）；
- 难以复用（其他项目想用某个函数要复制粘贴）；
- 难以测试（改一个地方可能影响全局）。

模块化把代码按功能拆到多个文件（模块），每个模块职责单一，通过 import 互相调用。

7.2.2 import 机制

```
# 方式1：导入整个模块
import math
math.sqrt(16)   # 4.0

# 方式2：导入模块中的特定函数/类
from math import sqrt, pi
sqrt(16)        # 4.0（不需要math.前缀）
pi              # 3.14159...

# 方式3：导入并起别名
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 方式4：导入模块中所有内容（不推荐，可能命名冲突）
from math import *
```

> 本项目统一用 `import numpy as np`、`import pandas as pd`、`import matplotlib.pyplot as plt`，这是数据科学领域的惯例别名。

7.2.3 模块搜索路径

Python 导入模块时，按以下顺序搜索：
1. 当前目录（脚本所在目录）；
2. 环境变量 PYTHONPATH 中的目录；
3. 标准库目录；
4. site-packages（pip 安装的第三方包）。

可以用 `sys.path` 查看搜索路径：

```
import sys
print(sys.path)
```

> 如果 import 自己的模块失败，检查：①模块文件是否在当前目录或 sys.path 中；②文件名是否与标准库/第三方包重名（如不要命名为 `math.py` 、 `pandas.py` ）。

7.2.4 包（Package）

包是包含 `__init__.py` 的目录，用于组织多个模块：

```
project_code/
├── __init__.py       # 标记这是一个包
├── config.py         # 配置模块
├── utils.py          # 工具函数模块
└── data_clean.py     # 清洗模块
```

导入包中的模块：

```
from project_code import config
from project_code.utils import haversine
```

> Python 3.3+ 支持隐式命名空间包（不需要 `__init__.py` ），但显式创建 `__init__.py` 仍是最佳实践，可以在其中写包级别的初始化代码。

7.2.5 相对导入 vs 绝对导入

- **绝对导入**：从项目根目录开始， `from project_code.utils import haversine` ；
- **相对导入**：用 `.` 表示当前包， `from .utils import haversine` 。

> 本项目用绝对导入，清晰直观。运行脚本时确保项目根目录在 sys.path 中（或在项目根目录下运行）。

7.2.6 requirements.txt

`requirements.txt` 列出项目依赖的第三方包及版本，便于在其他环境复现：

```
numpy>=1.24
pandas>=2.0
matplotlib>=3.7
```

生成和安装：

```
# 生成（把当前环境的所有包列出来）
pip freeze > requirements.txt

# 安装（在新环境中）
pip install -r requirements.txt
```

> `pip freeze` 会列出所有包（包括间接依赖），可能很长。可以手动精简只列直接依赖。

7.3 项目实现步骤

1. 创建 `project_code/` 包目录和 `__init__.py`；
2. 把配置写到 `config.py`；
3. 把工具函数写到 `utils.py`；
4. 在其他模块中 import 使用。

7.4 完整代码（模块化演示）

```
# ============================================================
# 文件1：project_code/config.py
# 功能：集中管理所有配置参数
# ============================================================

# 数据路径
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统(\09.交通时空大数据分析系统\B-项目数据"
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 分析参数
TARGET_DATE = "20181003"        # 目标日期
SAMPLE_ROWS = 20000             # 抽样行数（None表示全量）
MIN_RECORDS_PER_USER = 20       # 单用户最少记录数

# 算法阈值
LOW_FREQ_THRESHOLD = 4          # 低频点阈值（出现次数<此值）
TRAVEL_SPEED_KMH = 8.0          # 出行速度阈值（km/h）
K_MIN, K_MAX = 2, 10            # 聚类簇数范围
STAY_DIST_M = 1000.0            # 驻留距离阈值（米）
STAY_TIME_S = 2 * 3600          # 驻留时间阈值（秒）
PURPOSE_MATCH_M = 1000.0        # 目的匹配距离（米）
MODE_MATCH_M = 500.0            # 方式匹配距离（米）
```

```
# ============================================================
# 文件2：project_code/utils.py
# 功能：公共工具函数
# ============================================================

import numpy as np

def haversine(lng1, lat1, lng2, lat2, unit="km"):
    """计算两点间的球面距离（Haversine公式）。

    Args:
        lng1, lat1: 第一点经纬度
        lng2, lat2: 第二点经纬度（支持标量或数组）
        unit: "km"或"m"

    Returns:
        距离（km或m）
    """
    # 角度转弧度
    lng1, lat1, lng2, lat2 = map(np.radians, [lng1, lat1, lng2, lat2])
    # Haversine公式
    a = np.sin((lat2 - lat1) / 2) ** 2 + \
        np.cos(lat1) * np.cos(lat2) * np.sin((lng2 - lng1) / 2) ** 2
    d = 6371.0 * 2 * np.arcsin(np.sqrt(a))   # 地球半径6371km
    return d * 1000 if unit == "m" else d


def setup_chinese_font():
    """配置Matplotlib中文字体。"""
    import matplotlib.pyplot as plt
    plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei"]
    plt.rcParams["axes.unicode_minus"] = False
```

```
# ============================================================
# 文件3：test_import.py（在项目根目录运行）
# 功能：演示模块化导入
# ============================================================

import sys
sys.path.insert(0, r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版")

from project_code.config import TARGET_DATE, SAMPLE_ROWS
from project_code.utils import haversine

print(f"目标日期：{TARGET_DATE}")
print(f"抽样行数：{SAMPLE_ROWS}")

# 测试Haversine距离
d = haversine(123.41, 41.80, 123.43, 41.80)
print(f"两点距离：{d:.3f} km")
```

7.5 运行结果与解读

运行 `test_import.py`，应看到配置参数和距离计算结果。这证明模块化导入成功，config 和 utils 可以被其他模块复用。

7.6 当堂练习

1. 在 `utils.py` 中添加一个函数 `parse_time(time_str)`，把 "20181003093000" 转成 datetime 对象；
2. 在 `config.py` 中添加一个配置 `HEATMAP_WINDOW_MIN = 10`（热力图窗口分钟数），并在另一个脚本中导入使用；
3. 思考：如果 `utils.py` 导入了 `config.py`，`config.py` 又导入了 `utils.py`，会发生什么？（循环依赖）

7.7 课时小结

- 模块化 = 按功能拆文件，import 互相调用；
- 包是含 `__init__.py` 的目录；
- 配置集中到 config.py，工具函数集中到 utils.py；

- requirements.txt 管理依赖。

课时8：第一个可运行脚本

8.1 课时目标

写出第一个"读 CSV → 统计 → 保存"的完整脚本，建立工程习惯。

8.2 知识点深度讲解

8.2.1 脚本的标准结构

一个规范的 Python 脚本结构：

```
# ============================================================
# 模块文档字符串（说明这个脚本做什么）
# ============================================================
"""
脚本名称：first_script.py
功能：读取原始数据，统计行数和用户数，保存结果
作者：XXX
日期：2026-XX-XX
"""

# 1. 导入模块（标准库→第三方→本地）
import os
import pandas as pd

# 2. 常量/配置
DATA_DIR = r"..."
OUTPUT_DIR = r"..."

# 3. 函数定义
def load_data(path):
    """读取数据。"""
    pass

def analyze(df):
    """分析数据。"""
    pass

def save_result(result, path):
    """保存结果。"""
    pass

# 4. 主函数
def main():
    """主流程。"""
    df = load_data(...)
    result = analyze(df)
    save_result(result, ...)
```

```
# 5. 入口（只有直接运行时才执行main，被import时不执行）
if __name__ == "__main__":
    main()
```

> `if __name__ == "__main__":` 是 Python 脚本的标准入口。直接运行 `python script.py` 时 `__name__` 等于 `"__main__"`，执行 main；被其他模块 import 时 `__name__` 等于模块名，不执行 main。

8.2.2 "读-算-写"工程习惯

数据处理脚本的基本流程：

1. **读**：从文件/数据库读取原始数据；
2. **算**：清洗、转换、分析、统计；
3. **写**：把结果保存到文件/数据库。

每一步都要：检查输入是否正常、记录中间结果、处理异常。

8.3 项目实现步骤

1. 读取原始数据（前 1000 行）；
2. 统计总行数、不同用户数、时间范围；
3. 把统计结果保存到 CSV；
4. 打印运行日志。

8.4 完整代码（第一个可运行脚本）

```
# ============================================================
# 脚本名称：first_script.py
# 功能：读取原始数据，统计基本信息，保存结果
# 运行方式：python first_script.py
# ============================================================

import os           # 标准库：文件系统操作
import pandas as pd # 第三方：数据处理

# ------------------------------------------------------------
# 配置
# ------------------------------------------------------------
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统(\09.交通时空大数据分析系统\B-项目数据"
OUTPUT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
NROWS = 1000   # 只读前1000行做演示

# 原始数据10列字段名
RAW_COLS = ["timestamp", "imsi", "lac_id", "cell_id", "phone",
            "timestamp1", "tmp0", "tmp1", "nid", "npid"]


def load_raw_data(data_dir, nrows=None):
    """读取原始数据。

    Args:
        data_dir: 数据目录
        nrows: 读取行数（None表示全量）

    Returns:
        DataFrame
    """
    path = os.path.join(data_dir, "原始数据.csv")
    print(f"[读取] {path} (nrows={nrows})")
```

```
    df = pd.read_csv(
        path,
        header=None,                    # 无表头
        names=RAW_COLS,                 # 手动指定列名
        dtype={"imsi": str, "lac_id": str, "cell_id": str, "phone": str},  # 关键列按字符串读
        nrows=nrows,                    # 读取行数
        encoding="utf-8"                # 编码
    )
    print(f"[读取] 完成，共 {len(df)} 行，{df.shape[1]} 列")
    return df


def analyze(df):
    """分析数据基本信息。

    Args:
        df: 原始数据DataFrame

    Returns:
        dict: 统计结果
    """
    print("[分析] 开始统计...")

    # 1. 基本统计
    total_rows = len(df)
    unique_users = df["imsi"].nunique()

    # 2. 时间范围（timestamp是毫秒时间戳）
    ts_min = pd.to_datetime(df["timestamp"].min(), unit="ms")
    ts_max = pd.to_datetime(df["timestamp"].max(), unit="ms")

    # 3. 各列缺失值统计
    missing = df.isnull().sum().to_dict()

    # 4. lac_id分布（Top5）
    lac_top5 = df["lac_id"].value_counts().head(5).to_dict()
```

```
    result = {
        "总行数": total_rows,
        "用户数": unique_users,
        "最早时间": str(ts_min),
        "最晚时间": str(ts_max),
        "缺失值": missing,
        "lac_id Top5": lac_top5,
    }

    print(f"[分析] 总行数={total_rows}, 用户数={unique_users}")
    print(f"[分析] 时间范围: {ts_min} ~ {ts_max}")
    return result


def save_result(result, output_dir):
    """保存统计结果到CSV。

    Args:
        result: 统计结果字典
        output_dir: 输出目录
    """
    os.makedirs(output_dir, exist_ok=True)   # 确保目录存在
    path = os.path.join(output_dir, "first_stats.csv")

    # 把字典转成DataFrame（key为指标，value为值）
    # 注意：缺失值和lac_id Top5是嵌套字典，需要转成字符串
    flat = {}
    for k, v in result.items():
        if isinstance(v, dict):
            flat[k] = str(v)   # 嵌套字典转字符串
        else:
            flat[k] = v

    df_out = pd.DataFrame([flat])   # 一行DataFrame
    df_out.to_csv(path, index=False, encoding="utf-8-sig")
    # utf-8-sig编码带BOM，Excel打开中文不乱码
    print(f"[保存] 结果已保存到 {path}")
```

```
def main():
    """主流程：读→算→写。"""
    print("=" * 50)
    print("第一个可运行脚本：数据基本统计")
    print("=" * 50)

    # 1. 读
    df = load_raw_data(DATA_DIR, nrows=NROWS)

    # 2. 算
    result = analyze(df)

    # 3. 写
    save_result(result, OUTPUT_DIR)

    print("=" * 50)
    print("完成！")
    print("=" * 50)


# 脚本入口
if __name__ == "__main__":
    main()
```

8.5 运行结果与解读

运行 `python first_script.py`，应看到：
- 读取日志（文件路径、行数）；
- 分析结果（总行数、用户数、时间范围）；
- 保存日志（输出文件路径）；
- result 目录下生成 `first_stats.csv`。

> 用 `utf-8-sig` 编码保存 CSV，Excel 打开时中文不会乱码（BOM 头告诉 Excel 这是 UTF-8）。

8.6 当堂练习

1. 修改脚本，统计每个用户的记录数，找出记录最多的前 10 个用户；

2. 在 `analyze` 函数中添加"按小时统计记录数"的功能；
3. 把脚本改成支持命令行参数（如 `python first_script.py --nrows 5000`），提示：用 `sys.argv` 或 `argparse`。

8.7 课时小结

- 脚本标准结构：文档字符串 → 导入 → 配置 → 函数 → main → 入口；
- "读-算-写"是数据处理的基本流程；
- `if __name__ == "__main__":` 是标准入口；
- 保存 CSV 用 `utf-8-sig` 编码，Excel 打开中文不乱码。

三、本日小结

1. **项目认知**：信令数据 = 谁+何时+在哪，本项目 6 大模块从清洗到可视化；
2. **数据全貌**：7 个数据文件，两个关键坑（字符串读取、GBK 编码）；
3. **环境搭建**：Python 3.10+ + pip 换源 + 三件套安装验证；
4. **开发工具**：Jupyter Notebook（探索）+ VS Code（工程）；
5. **Python 基础**：动态类型、缩进、for-each、函数、异常、数据结构、推导式、文件读写；
6. **模块化**：config.py 集中配置、utils.py 公共函数、import 机制；
7. **工程习惯**：读-算-写流程、脚本标准结构、`if __name__ == "__main__"`。

四、本日作业

1. 在自己电脑上搭好环境，运行 `check_env.py` 确认三件套可用；
2. 运行 `first_script.py`，把 `nrows` 改成 10000，观察统计结果变化；
3. 写一个脚本，读取基站经纬度数据，统计经纬度范围，确认在沈阳范围内；
4. 预习 Day02：思考"为什么 Python 循环慢，NumPy 向量化快"。